Spec-Driven Development (SDD)

Evolución hacia Spec-Driven Development (SDD)

El **SDD** es una **evolución** de metodologías como **TDD** (Test-Driven Development) y **BDD** (Behavior-Driven Development), optimizada para **agentes de codificación** (AI Coding Agents). En este modelo, una **especificación** no es un documento de texto muerto, sino un **archivo estructurado** (Markdown, YAML o DSL) que describe explícitamente:

1. Intención del usuario (User Journey).
2. Restricciones técnicas (Tech Stack, Patrones).
3. Criterios de éxito (Pruebas/Validaciones).

La Premisa Central: LLMs y la Especificación Rigurosa

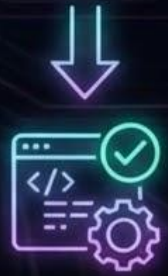
Prompts Vagos ("Vibe Coding")

Los LLMs son buenos completando patrones, pero malos "leyendo la mente".

Resultado: Código que “parece” correcto pero falla en casos borde.

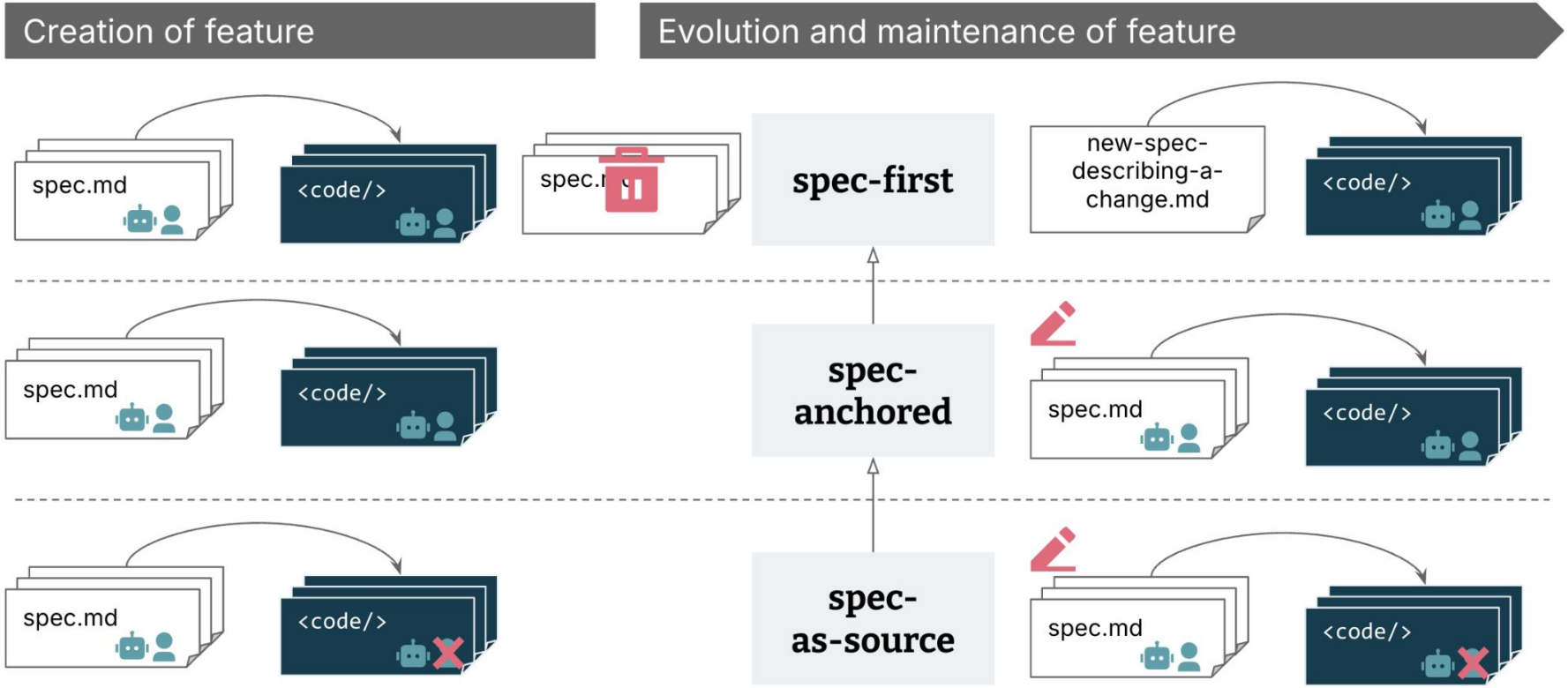
Especificación Rigurosa (SDD)

Proporciona instrucciones explícitas y detalladas.



Resultado: Los LLMs actúan como programadores deterministas.

Levels of “spec-driven”



https://martinfowler.com/articles/exploring-gen-ai.html

El Surgimiento del SDD: Respuesta a los Riesgos del 'Vibe Coding'

El SDD responde a los riesgos del 'Vibe Coding' (escribir código basado en 'vibras' o intuiciones rápidas con IA). Aunque rápido para prototipos, falla en producción por:

- **Alucinaciones y Deriva:** Sin un contrato claro, la IA inventa APIs o ignora reglas de negocio.
- **Deuda Técnica:** El código generado suele ser 'un solo uso' y difícil de mantener si no hay una estructura arquitectónica previa.
- **Falta de Contexto:** Una IA no conoce tus estándares de seguridad o arquitectura a menos que se especifiquen explícitamente.

El SDD permite separar el **'Qué'** (estable, definido en la spec) del **'Cómo'** (flexible, generado por la IA), facilitando la regeneración de código sin perder la lógica de negocio.

Los 4 Pasos del Flujo SDD

Basado en herramientas modernas como **Spec Kit** o **Currante**, el flujo de trabajo se estructura así:

1. Especificar (Specify)

En lugar de escribir código, describes el problema de alto nivel.

Input:
"Necesito un sistema de gestión de tareas donde los tengan roles (admin/viewer)."

Acción del Agente:
La IA genera un archivo spec.md detallando historias de usuario, flujos de datos y criterios de aceptación.

Rol del Humano:
Revisar y refinar la intención. Este es el documento vivo.

2. Planificar (Plan)

Aquí se definen las **decisiones arquitectónicas**.

Input:
"Usa Python con FastAPI, arquitectura hexagonal y PostgreSQL."

Acción del Agente:
Genera un plan.md técnico. Decide estructuras de carpetas, firmas de APIs, esquemas de base de datos y estrategias de prueba.

Validación:
El arquitecto verifica que el plan cumpla con los atributos de calidad (seguridad, rendimiento) antes de escribir una sola línea de código.

3. Desglosar (Tasks)

El plan se rompe en **unidades atómicas.**

Input:
Plan técnico (plan.md) con decisiones arquitectónicas.

Acción del Agente:
Convierte el plan en una lista de tareas pequeñas e implementables (ej: "Crear modelo de datos", "Crear endpoint de login", "Escribir test de integración").

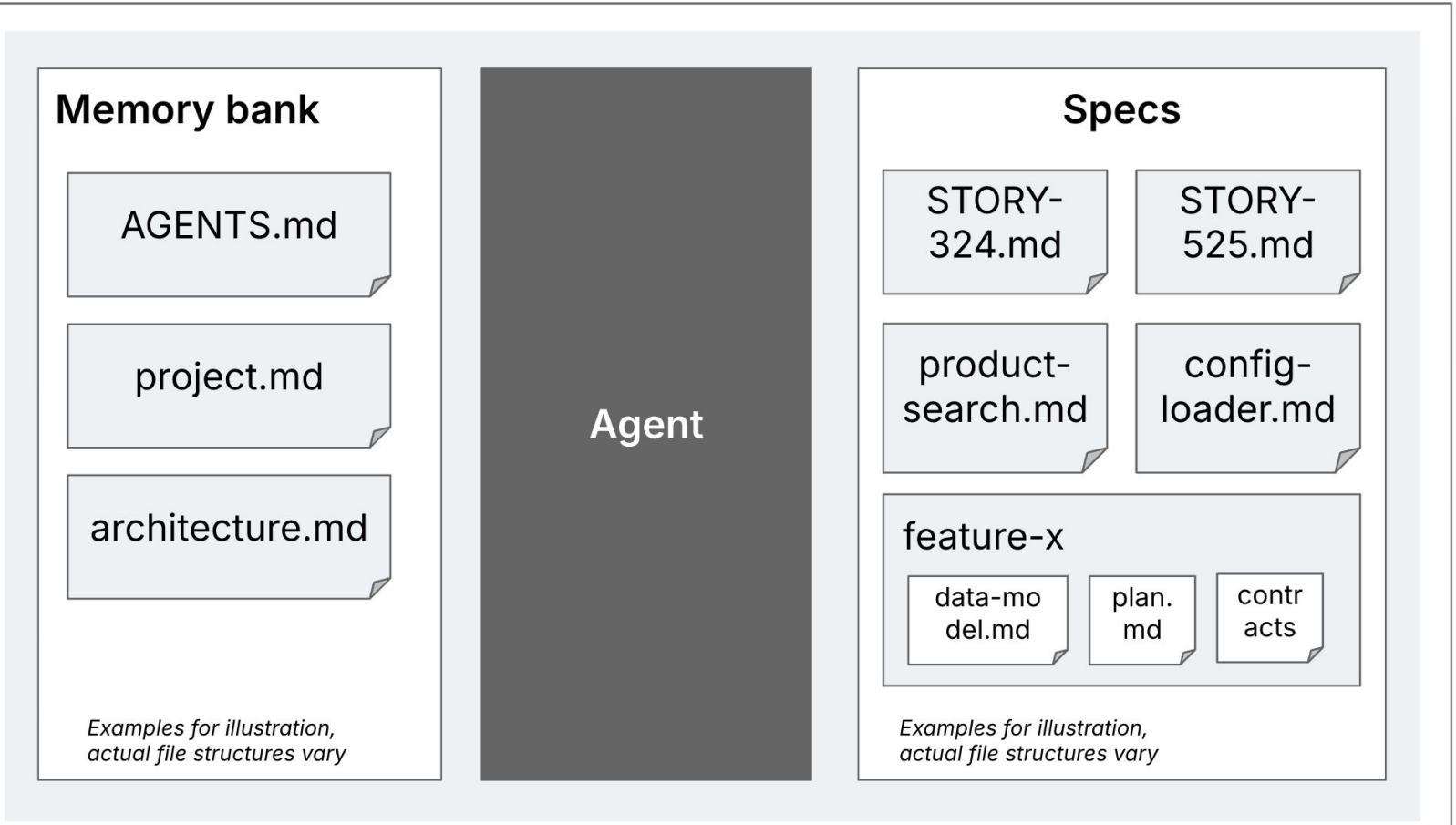
Objetivo:
Esto permite que la IA trabaje en bucles cortos de retroalimentación, similar a TDD.

4. Implementar y Verificar (Implement)

El agente ejecuta las tareas **una por una**.

Acción:
La IA genera el código y los tests basándose estrictamente en la especificación y el plan.

Rol del Humano:
No revisas “dumps” masivos de código, sino que verificas que la tarea cumpla la spec. Si falla, no corriges el código manualmente, corriges la especificación o el plan y regeneras.



https://martinfowler.com/articles/exploring-gen-ai.html